

Decision-OS V13: From Human Aspiration to Working AI Loops

Design and Field Evidence from LoopKit and Its Derivatives

Shinichi Nagata

September 17, 2026

English edition / September 17, 2026

This paper aims to become the reader’s next 1.01. Here, 1.01 means a change that improves the conditions for subsequent work without impairing the people or resources that sustain it; it does not mean a numerical improvement of 1%. We call the long-term direction held by a human *Aspire*. This paper offers material for designing the scope of work delegated to AI, the verification required, and the judgments retained by humans. It aims to help readers identify what to establish first for their own *Aspire*, what to entrust to AI, and what to carry forward, so that they can take the parts relevant to their own work. It also provides entry points for trying the public implementation and tracing decisions, changes, and external outcomes.

Abstract

After being given multiple *Aspires*, can AI identify gaps and select both the next task and the conditions needed to execute it? This paper retrospectively examines the design and operation of Decision-OS V13 LoopKit. An autonomous trial in July counted an intake entry point that was not yet usable as progress; an independent evaluation classified the trial as a failure. Subsequent changes bundled the authority needed to reach a usable state before execution and prohibited silently substituting easy internal work because authority was missing. The records also trace repairs supported by principles selected for another AI, inheritance by the derivative operation Value-Locked, the merging of 32 of 54 public pull requests across 28 repositories, and the sharing of lessons across projects. However, humans continued to supply new goals and instructions, and neither overall autonomy nor an improvement in success rates has been established. Through implementation, change histories, and external outcomes, this paper shows what was entrusted to AI, what was retained as learning, and how subsequent choices were revised.

1 What the Principles Became in Practice

The author’s motivation was to “build the conditions under which AI could be trusted,” while recognizing that AI can make mistakes. [10] Through practice, the work sought to establish the grounds for delegation, the conditions for returning decisions to a human, and what to change after failure.

Against the aim of autonomous progress from *Aspire*, this paper examines how far task selection, execution, and the transfer of experience could be entrusted to Decision-OS V13 LoopKit, and where human explanation and judgment remained necessary.

LoopKit’s public repository contains decision principles given to AI, documents recording the current state and decision rationale, and code that checks conditions for continuation. These components do not constitute a single fully automated execution program. A human specifies direction and the authorized scope, and AI uses the principles and records relevant to the purpose to choose a next action. It preserves artifacts, decision rationale, unresolved issues, and restart conditions, and selectively passes relevant material to derivative operations. [1,3]

The author reports that, although direction and procedures initially required detailed explanation, individual instructions could increasingly be shortened in Value-Locked by relying on accumulated decision principles and records. We examine the author’s observation that the burden of explanation and step-by-step intervention decreased relative to the amount accomplished, together with the operating conditions that supported delegation. [10] In addition to this operator observation, records describe a subsequent AI receiving selected grounds for action and carrying out a repair without repeating the earlier selection or completed audits. [11]

In Value-Locked’s contributions to external open-source software (OSS), 54 public pull requests (PRs) were created from Shin’s GitHub account ([shin4141](#)) between August 14 and September 4, 2026. At the September 16 observation, 32 had been merged across 28 repositories. [2] External acceptance is a concrete outcome, but it does not by itself demonstrate that LoopKit improved efficiency or success rates. This paper examines how prepared principles and records were used in execution and what was carried forward.

This is a retrospective case study of related projects involving one developer and AI. Its sources include evidence extracted from the original note and implementation, candidate comparisons and PR change histories, Value’s evidence packets and a timeline of 54 PRs, and review and handoff records from plugin validation. [1–5,11,23] The author’s recollections on September 16–17, 2026, concerning design intent and changes in burden are distinguished from confirmation through contemporaneous records. [10] The following sections describe the design, use in the core project, Value’s practical work, and sharing across projects. Section 8 connects these observations to readers’ own use.

2 Intended Autonomy and the Components Supporting Operation

Hereafter, LoopKit is called the “core” (called “13” in the records), and the derivative operation Value-Locked is called “Value.” The core components have the following roles. [1,3,11]

Component	Role and terminology used in this paper
<code>AGENTS.md</code>	Operating rules for AI and the conditions for consulting material needed for the current decision
<code>field_notes/</code>	Field Notes (FN) recording decision rationale, failures, and principles; for example, FN021 concerns required intermediate milestones and FN025 consolidates reporting formats
<code>handoff/current_codex_handoff.md</code>	A handoff recording the core’s current state, next responsible actor, authorized scope, and restart conditions; derivative projects may use different filenames
<code>docs/context_compression.md</code>	A documented operating policy for compression that preserves the information needed to resume work

Component	Role and terminology used in this paper
Stage C in <code>decision_os/companion/small_compound_loop.py</code>	Code that checks execution results, remaining requirements, authority, and continuation conditions for fixed goals and requirements

A Gate is a decision that assigns GO, HOLD, CAP, or BLOCK to execution or continuation; it is not the name of a separate component. Written instructions to AI are distinguished from the conditions that code can check.

2.1 Aspire and Judgments Retained by Humans

Aspire extends beyond the completion of a single task and provides a basis for deciding what constitutes a beneficial change at a given time. Carrier refers to the people, resources, and capacity for recovery that sustain continuation. Re-entry capability refers to the conditions that allow work to be resumed when needed. [1,3]

The author initially sought an operation in which, once multiple Aspires were specified, AI would identify gaps, choose investigations or necessary requests to humans, and advance without step-by-step instructions. [10] In practice, a route remained in which Shin and a dialogue AI prepared the next instruction and passed it to Codex. The foundations for continuation and the division of judgment also changed as this discrepancy between the intended and actual operation was addressed. [23]

Seat is the human authority to decide direction, value, and the authorized scope. AI takes responsibility for comparing candidates and choosing execution methods within that scope. If every matter resolvable under existing conditions is returned to a human, human attention becomes a constraint on continuation. Decisions that must return to humans are therefore distinguished from operational judgments assigned to AI.

2.2 Foundations Established for Continued Work

The author describes having developed the following foundations before LoopKit. [10] Corresponding documents and implementations also appear in the collected material. The table describes their intended roles. [1,3]

Foundation	Role in sustaining continuation
Completion and stopping conditions	Check what has been achieved and stop continuation without progress or beyond the permitted scope
Recording and rereading trajectories	Allow a return to earlier decision rationale, assumptions, and unresolved problems
Handoffs	Enable the next responsible actor to receive the current state and restart conditions
External memory	Retain relevant experience beyond a single conversation
Reducing context and token burden	Limit repeated reading while preserving the information needed to resume

The author recalls that handoff exchanges, in which a new chat asked about missing information and the previous chat supplied it, imposed a burden on him as the intermediary and altered the

tone of the conversation. He therefore shifted toward recording trajectories: preserving decision rationale and rejected alternatives, as though "placing points" along the project's course, so that a subsequent chat could trace the history. This design sought to approximate continued collaboration with the same counterpart while recognizing that individual identity could not be preserved. [10] The reduction in repeated explanation has not been measured.

Adding stops and checks alone does not advance execution. The records also retain the author's observation that V13 tended to prioritize prevention, along with measures against excessive stopping. Some of these remain unvalidated specifications. [3,10] The design therefore includes the ability to choose actions within the conditions that must be protected.

2.3 Passing Each Child What It Needs

An operation derived from the core to serve a distinct purpose is called a "child." The author explains that he had the AI working on the core read earlier Decision-OS papers and select the parts relevant to each child's purpose. He also recalls that many principles were already present in the operation, so newly incorporated material was limited. [10] The scope of material read is not the same as the scope newly adopted. Value has explicit receipt records for Seat, Gate (execution and continuation decisions), Carrier, Completion, reconnection, and handoffs. [2,3] This does not mean that the selection process for every child has been recovered.

When a child is established, the inherited principles, completion conditions, and experience to carry forward are specified. Rules added through the child's own work are distinguished from inheritance from the parent and are not applied retroactively.

2.4 1.01 as the Next Change Chosen on These Foundations

In the original note, 1.01 represents a loop that improves future conditions without impairing the Carrier. [1] At selection, a change is a candidate improvement; after execution, what actually changed must be checked.

The historical selection method placed a path between Aspire and the current state and selected the first missing required intermediate milestone on that path. [3] The author now explains that he wanted to avoid fixing the route too early, instead preparing elements likely to be needed across multiple routes while retaining alternatives. [10] Something required on one path is not necessarily required on every path. Assumptions should be retained, exploration and alternative routes compared, and the selection method itself reconsidered.

In operation, benefits, costs, risks, reversibility, future options, and doing nothing are compared, then translated into a next action together with completion conditions, scope, authority, and stopping conditions. [3] This does not require a lengthy comparison report every time.

2.5 The Boundary Between Operating Principles and Code

The author sought to give Codex a minimal set of rules for choosing the next task. He recalls that Field Notes initially served as a place for ideas, and that AI began to identify common features and relationships in accumulated entries and connect them to higher-level rules. [10] The records contain changes that incorporate individual failures into contracts and consolidate overlapping reporting rules. [23] These operating principles can be added or removed; their role differs from that of code checking continuation.

Stage C checks stored results and authority for a predefined Goal and one to three requirements, then converts the first unmet requirement into the next Task. It does not generate a semantically

optimal goal from an unrestricted Aspire. Human and AI judgment is still needed to prepare the content and ordering of requirements.

Figure 1 shows how decision criteria and records are reused to support short instructions, and how decisions return to a human.

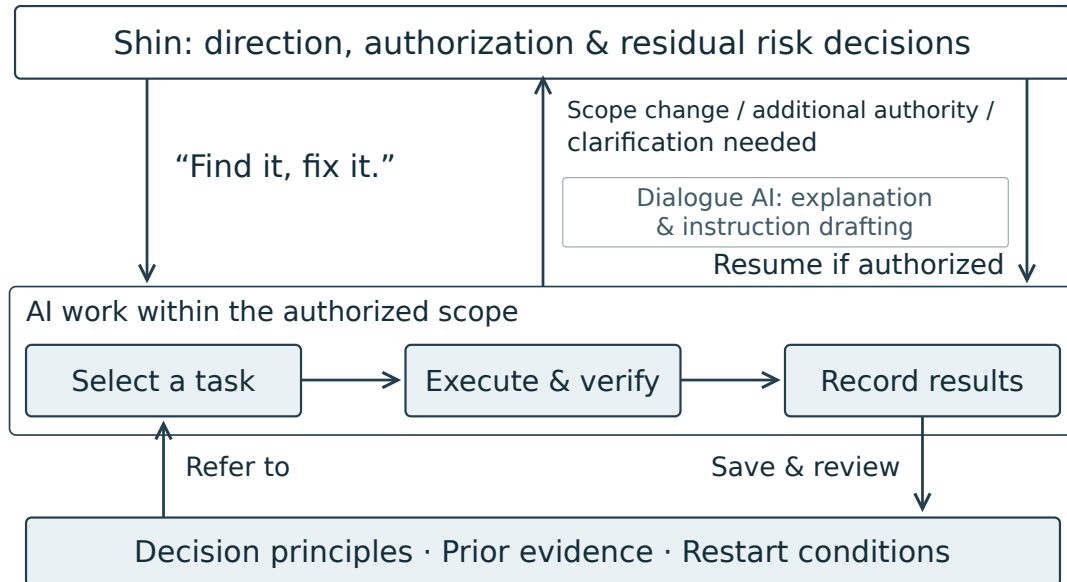


Figure 1: The structure supporting short instructions. The arrows represent information and judgment relationships organized in this paper; they do not indicate an automated implementation or an assignment of roles common to all cases. The dialogue AI supports explanation and instruction drafting. Work resumes only when the necessary authorization is present (Section 2 and Sections 4.2–4.5).

3 What Was Selected and Changed in the Core

3.1 Testing Autonomous Progress and Repairing the Gap to Actual Effect

The first self-application record, dated June 11, 2026, set out a policy of checking stopping and resumption through bounded use. [11,16] This was the initially implemented scope of use in relation to the broader aim of autonomous progress from Aspire.

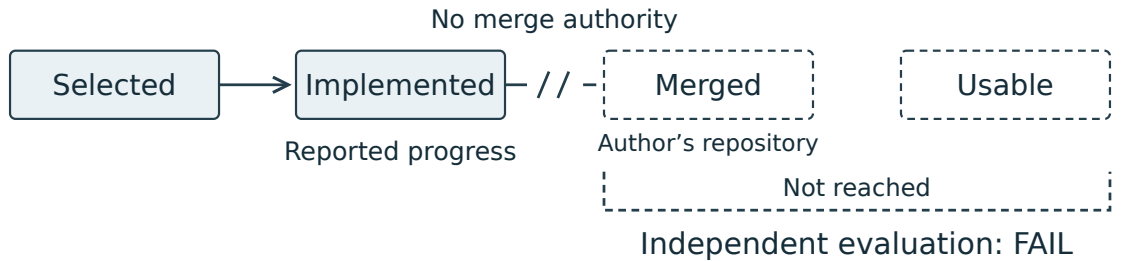
The autonomous trial on July 23 prioritized revenue and selected a dedicated issue template and an accompanying link for inquiries from prospective buyers. However, the implementation remained on an unmerged branch and was not yet an entry point that users could access. The trial lacked merge authority. An evaluation by another Codex did not accept the first action as progress, contrary to the implementer’s assessment, and classified the entire Run as FAIL. [23, G07–G11]

Subsequent PRs bundled the authority needed for execution and integration before the start of a bounded Run. After Shin’s explicit authorization, PR #7 in the core’s own repository integrated the entry point into the default branch. This was integration on the author’s side, distinct from acceptance by external maintainers in Section 4. A further rule required assessing candidates’ relative value before filtering them by available authority, and prohibited silently substituting easy internal work because authority was missing. [23, G12–G14] The confirmed outcome was the availability of the entry point, not use by buyers or revenue. The later correction does not reclassify the initial failure as a success.

This sequence showed the need to check, at selection time, both what to choose and the execution conditions needed to achieve the intended effect. Overall autonomy remained unachieved: human-side instructions still initiated implementation and integration.

Figure 2 shows how the scope of verification changed between the July 23 trial and the July 24 revision.

July 23: initial trial



July 24: revised execution

Required permissions bundled → Authorized by Shin

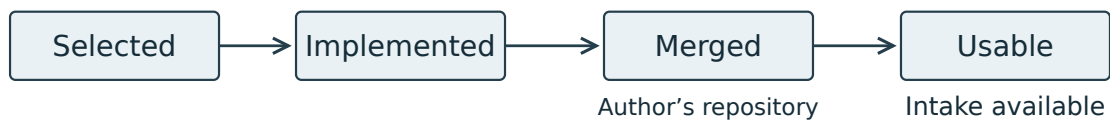


Figure 2: The difference between building something and making it usable. The initial FAIL was retained while the scope of verification expanded from implementation to the availability of the intake entry point. The conditions required by a valuable candidate were checked, rather than silently substituting another internal task because authority was missing. What was confirmed was the availability of an entry point in the core’s own repository, distinct from customer use, revenue, or acceptance by an external maintainer (Section 3.1; reference [23]).

3.2 Passing Selected Grounds for a One-Line Repair

The June 12 compression policy stated, “Compress the context, not the ability to restart.” [17] The design preserved the current situation, completion status, next permissions and prohibitions, known errors, and unverified matters. A subsequent stored report described another chat identifying and executing the remaining action without using the entire preceding conversation. [11, S03–S06] The handoff itself had become longer. This is an example of restart information being used, not evidence of a token reduction rate.

A July 13 case traces how earlier principles led to a one-line repair by another AI. The problem was that the recipient’s handoff still listed already completed work as the next action. [11, S17–S20; 12,13]

Stage	What happened in this case
Problem in the current state	The next action in the handoff pointed to completed work and therefore disagreed with the actual state
Selection by the first AI	It selected four existing principles—required intermediate milestones, separating completion from the next authorization, re-entry, and avoiding proliferating reports—and proposed a correction
Transfer to the next AI	The selected grounds and the task to be repaired were passed on

Stage	What happened in this case
Execution by the recipient	It changed one line in the handoff without repeating the earlier selection, experiments, or completed audits
Change retained	The stale next action was removed, and restart conditions placed new work on HOLD until authorization for its target and scope was in place

What was passed on was a repair instruction and the grounds supporting its rationale and scope. One of the four principles was used to avoid adding reports: inheritance did not mean importing every rule.

In addition to the evidence collector’s comparison of original text and Git diffs, the manuscript preparation process checked fixed versions of the public selection and execution reports.[11–13] Verification covered the transfer through this repair; it did not independently re-audit the private original logs or reproduce execution.

3.3 The Controls Themselves Were Repaired and Reduced

June records identified a burden caused by multiple reporting formats repeating the same judgments. In July, these were consolidated into a basic report with conditional additional information. This lesson was selected as one of the four principles in the preceding section and used to avoid bringing the entire reporting system into the recipient’s workflow.[11, S12/S17/S34]

On July 28, Handoff Acceptance Guard was repaired following an assessment that its custom proof format created excessive complexity and erroneous judgments of non-completion. Eight elements were removed: Work ID, ownership syntax, action signatures, completion predicates, boundary clauses, dedicated tokens, reference graphs, and issue classification dependent on a custom language. The repair returned to elements already used by the recipient. Seven elements were retained: safe file reading; stable input acquisition; stable repository-state acquisition; read-only Git observation; separation of current and historical state; deterministic output ordering; and handling that avoids inadvertently displaying input content in results.[14, §5]

Stored repair record	Details
Implementation size	Target module: 2,750 → 2,095 lines; target tests: 2,559 → 1,603 lines
Test inputs	Seven custom-format fixtures replaced by seven fixtures using the existing format
Repair burden	Two rounds of rework with review
Validation	Targeted tests: 66/66 PASS. The full suite initially retained seven errors caused by sandbox restrictions on localhost connections; rerunning the same command in an authorized environment yielded 310/310 PASS

Line counts measure a change in implementation size, not speed or maintenance cost. An independent repair review was pending at the time of the repair report, but a subsequent final record reports integration through PR #39.[14, §§6–9,13; 15, §§2,7–8] What was checked here was the stored report; the tests were not rerun.

Building and repairing controls has a cost. What was added, left unused, and removed should be considered together. Accumulating rules is not itself an increase in capability.

3.4 The Expanding Range of Recorded Outcomes

Even when total working time is unknown, the outcomes produced within the observation period can be described. The following timeline covers records recovered so far, not a public inventory of every derivative OSS project. It distinguishes internal development dates, subsequent use dates, and external acceptance dates. [2,5,11]

Period	Confirmed outcome	What the date represents
June 11–12	Bounded continuation and a restart handoff were established; use by another chat was reported	Stored design and use records
July 13	Four principles were selected, and another AI repaired the recipient's handoff	Proposal and subsequent execution records
July 23–24	The autonomous trial's FAIL was preserved; subsequent changes integrated the entry point into the core repository and checked necessary authority in advance	Trial, independent evaluation, and subsequent PR records [23]
July 28	A repair removing the custom validation format was implemented, tested, and integrated	Core repair and integration records
August 9	Principles selected for Value's purpose were installed and used to organize its actual state and handoff	Initial setup and use records for the derivative operation
August 10	AI compared candidates, selected, and implemented repairs to the entry README and stopping without progress	Core execution records 13-104 and 13-106
August 14–September 4	54 PRs were created for the external OSS projects in scope	PR creation period
September 10–11	Review comparisons and a selection from another project were returned to Value and passed to the next responsible actor	Storage and handoff records
September 16 observation	32 of the PRs had been merged across 28 repositories	External acceptance status at observation

Activities expanded from establishing the core to use in another context, derivative operations, external acceptance, and sharing across projects. For a timeline of all derivative OSS projects, the relationships among lineage, first operation, publication, and acceptance have not yet been fully checked. We do not calculate an overall growth rate or efficiency per hour from this limited table, or use repository creation dates or commit counts as proxies for workload.

3.5 Making the Core Understandable at the Entry Point Before Building a Demo

In core execution record 13-104 on August 10, a path was laid out from understanding the core's problem and minimal trial procedure at the public entry point to trying and adopting it. The README then contained 879 lines, and its opening described an optional Companion. The two decisions addressed by the core appeared much later. [3]

AI compared making a demo, reorganizing the entire README, external distribution, new

features, and doing nothing. It selected a change that placed an explanation of the core and its existing trial procedure on the first screen. A demo could aid understanding, but readers first needed to know what the product was. The record explicitly cites Field Note 021, and its use corresponds to the actual diff.

The repair also preserved references used by historical tests and affected three files. What was confirmed was that candidate comparison led to a concrete change and that the content at the entry point changed.

3.6 Detecting Normal Termination Without Progress

In core execution record 13-106, also on August 10, a repair was selected from six improvement candidates to address continuation to the next execution despite no new progress on declared requirements. The reasons were that progress could be judged from existing records, a test before the repair could demonstrate the gap, and the change could be bounded. [3]

Stored post-repair tests confirmed that execution stopped after the first run if no new requirement had been satisfied, and after the second run if that run made no progress. Conditions distinguishing normal termination from progress remained as code and regression tests. They did not measure useful changes that had not been declared or actual total costs.

On August 11, seven candidates and then nine candidates were compared. Even under a condition that gave no extra credit to safety-oriented options, two defects that could be repaired with small changes were selected: reuse of stale authorization and use of evidence from before a change within the same execution to judge completion. Inability to resume after interruption was assessed as important in both comparisons but was not selected. A later audit by another model, which was not shown the candidate list, also identified it. [23] Detecting a problem and selecting it for immediate repair are different; how to handle important deferred candidates remained unresolved.

3.7 Judgments Assigned to Humans and AI

Humans supplied the initial direction and execution scope, and AI compared candidates, implemented, and validated within that scope. Records 13-104 and 13-106 report zero Human Seat calls during execution. This does not mean that initial human authorization or assistance with the validation environment was absent. [3]

4 Value-Locked in Practice: Inherited Principles

4.1 Arranging Principles Around the Purpose

Value-Locked began on August 9, 2026, as a separate repository derived from V13. Its initial documents identify LoopKit as a source and place decision ownership, Gate, Carrier, Completion, reconnection, and handoff responsibility into local procedures. The August 27 Commercial Compound Loop records receipt of Field Notes 021–024 and translates the sequence from the current state to the first required missing element into a commercial goal. [2,3]

What was inherited was the set of principles needed for the purpose, not V13’s entire execution infrastructure. Nor are rules added on August 27 treated as causes of all earlier outcomes. Each case is read using the information and rules that existed at the time.

According to the author’s recollection, before searching for individual cases, Shin and the dialogue AI spent several days comparing the Decision-OS papers with past practical experiences.

They explored distinctions between states or judgments that could not be equated, and the failure boundaries arising when those distinctions were overlooked, then identified candidates they could address. [10]

The author explains that the decision premises established through this preparatory exploration were carried into subsequent work through trajectory records and handoffs. At execution time, a short instruction such as "Find it" could therefore reconnect to the search targets and decision criteria already shared. Work on deciding what problems to look for had preceded the point at which short instructions became sufficient. As lessons and merges accumulated in practice, they broadened the fields covered and also began seeking contributions to well-known projects. [10]

This exploration rested on an agreement with AI to follow existing rules. The author attributes the expansion of options to learning failure conditions and the responses they required, thereby increasing the range of boundaries they could address. He does not expect this paper to change the industry's rules, nor does he want readers' expanded activity to increase the burden on maintainers. [10] This paper likewise examines how individual contributions were achieved within each project's acceptance policy and the maintainers' judgment.

4.2 Minimal Repair Became More Concrete Through Experience

The idea of a repair that was necessary and sufficient existed from the outset. In practice, it developed into checks for competing work and responsible parties, the logical scope of a change's effects, review burden, handling unresolved cases, and reconnecting when events occurred. [2]

A repair with few changed lines is incomplete if it does not satisfy the required contract; conversely, even when many test-data values change, the specification change can remain bounded. Minimality is judged by the behavior and contract being changed as well as line count.

Case	Changes arising from judgment and review	Recorded external outcome
Anza / Solana Kit #1971	Used a maintainer preference expressed in an earlier PR to limit the change to a single field's type definition; revised type tests after further review	Merged August 27
Sudachi.rs #360	Waited for a Python support-policy decision, then separately verified rejection, support, and compatibility for the relevant versions	Merged August 28
PowerGridModel #1547	Revised the structure in response to review, removed unnecessary variation in expected-value data, and checked against an alternative calculation basis	Merged September 15

These cases trace the conversion of experience and feedback into concrete implementation and verification through external acceptance. [2] Neither the number of minutes saved in maintainer review nor whether any single change determined acceptance was measured.

There were also cases with different outcomes. Papra was closed because of competing work despite the use of existing knowledge, and some cases were stopped before submission after insufficient repairs were recognized. Use of knowledge, technical validation, and external acceptance are observed separately.

In the FiPy investigation on August 23, Codex used counterexamples to detect two remaining stale-update paths that the reporter's one-line fix would leave unresolved, and preserved the

validation conditions. [20–22] Before submission, the dialogue AI produced authorization wording before explaining the work. Shin stopped the process and requested an explanation of the changes, validation results, and untested scope. After receiving that explanation, he decided to accept the residual risk, required clarification of the boundary between instructions for Codex and the PR body, and authorized submission. [22, §6]

The submitted PR #1224 was closed because it did not comply with the AI contribution policy and imposed additional review burden. Four days later, Shin requested comparison with PR #1225, which a maintainer had independently created, and instructed Codex to rerun the saved counterexamples. Codex confirmed one stale-update path. After Shin reported it, the maintainer corrected part of the causal explanation, repaired the relevant code, and merged the change. [20–22] This is a case of preserved validation being used in a subsequent repair. It is not treated as adoption of #1224 or added to the 32 direct merges.

4.3 Roles, Outcomes, and the Observation Period

The population consists of 54 public PRs created from Shin’s GitHub account (**shin4141**) in repositories owned by others. The observation time is September 16, 2026, at 04:02:21 UTC. The denominator excludes self-owned repositories, candidates stopped before submission, private or deleted PRs, other identities, and submissions outside GitHub. [2]

In the collected material, Shin was responsible for execution scope, permission to publish, and final risk judgments. The execution AI/Codex was responsible for tracing sources, testing, inspecting diffs, organizing candidate lessons, and assembling evidence. [2, §4.4] The author recalls that, from some point onward, he did not check every step of execution; when human judgment was needed, he requested an explanation from the dialogue AI (“O”) and decided whether to authorize the action. [10] Responsibilities at each stage and the amount of human involvement were not recorded on a common scale for all 54 cases.

Status at observation	Count
Directly merged	32
Closed without a direct merge	8
Open	14
Total	54

The direct merges were accepted across 28 repositories. The eight closed PRs include two cases adopted through another PR or another route. These are not added to the 32, and not every closure is treated as a technical rejection. One of the 14 open PRs was a draft.

As an update after the fixed observation, the repair from OpenSSL PR #32685 was integrated into upstream **master** on September 16. This is confirmed by the maintainer’s acceptance comment and commit **aeeca5a9e07166183fe323f336c9177a9b524c78**, which records **shin4141** as author and the source PR in **Merged-from**. [24] The repair changes recursive seed-source construction from stack exhaustion to a failure that returns a **RAND** error. The GitHub API records the PR state as **closed** and **merged** as **false**, but integration of the repair upstream is confirmed. In this paper, “direct merges” counts PRs recorded as merged on GitHub, distinct from the full set of outcomes in which repairs were integrated upstream. Because #32685 was open at the fixed observation, the table above and the seven-day merge rates remain unchanged; it is treated as a subsequent external acceptance case.

The initial fixed cohort of 28 PRs had eight direct merges at the baseline observation of August 16, 01:03 JST (August 15, 16:03 UTC), increasing to 19 by September 16. PRs open early in

the observation period retained the possibility of later merging. This does not establish that waiting caused the outcomes, but it shows that an early lack of response cannot be classified as permanent failure. The baseline time was checked directly against the fixed record, and the corresponding eight PRs were also checked against the CSV. [18]

Appendix A, supplied as a separate supplement, extracts the 54 public PR URLs; creation, merge, and closure times; status at observation; and draft status. [19] It excludes internal learning IDs and private source indexes. Aggregates can be recalculated from the saved snapshot, but the data do not represent independent retrieval of every event.

4.4 No Improvement in Success Rate Was Confirmed

We compared the rate of merging within seven days of PR creation before and after the recording of operating rules. The dividing point was the commit at August 15, 2026, 12:46:30 UTC that fixed bounded repair and mandatory preflight. All 54 PRs had at least seven days of observation. [2]

Group	PRs	Merged within seven days	Rate
Before the rules were fixed	28	18	64.3%
After the rules were fixed	26	11	42.3%

This result does not support the hypothesis that success rates rose in the later period. An increase in cumulative outcomes is distinguished from an improvement in success rate within a common observation window.

Case composition, review conditions, models, and validation methods also changed between the periods, so this comparison does not estimate the causal effect of the rules. Minimal repair predated the dividing point, and the rules continued to change afterward. We do not explain the result by using organizational prominence as a proxy for difficulty.

4.5 Accumulated Preparation and the Division of Judgment Behind Short Instructions

The short instruction "Find it, fix it" presupposed existing decision principles, authorized scope, tests, records, and conditions for returning decisions to humans. Regarding the preparatory exploration in Section 4.1, the author explains that carrying forward the decision premises developed over several days through trajectories and handoffs enabled shorter subsequent instructions. [10] Compression in this account means retaining the necessary premises in a form the next responsible actor can use again, rather than merely shortening instruction text. Assessing brevity therefore requires including the effort to construct, update, and pass on those premises.

Shin reduced the burden of checking every execution step, while pausing decisions and asking for explanations when the explanation was insufficient (Section 4.2). The execution AI handled implementation tests and diff inspection (Section 4.3). Step-by-step monitoring, reading code, and deciding on submission are different forms of involvement; the records do not establish that the same checks were performed in all 54 cases. Transferring only the short instructions cannot be assumed to reproduce the same operation.

The burden reduction introduced in Section 1 is the author's observation, not a comparative measurement controlling for conversation volume or time. [10] Reuse of decision criteria and

memory could plausibly reduce repeated explanation, but its contribution has not been isolated. The author’s account includes both short search instructions and explanation and authorization where understanding was required. Assessment of reduced explanation burden should also check whether informed judgment was preserved.

Future measurement should include human explanation time, the number and content of interventions, repeated explanations, and repair work, alongside outcome counts. Fewer interventions alone should not count as improvement: necessary human judgment should be retained, and unverified completion claims should not increase.

5 Returning Judgment to Another Project

5.1 Test Gaps Exposed in the Actual Environment

In O-69, the pre-submission validation of Impact Trajectory Companion, local tests and an independent audit were passed, but testing in actual ChatGPT subsequently reported 3 PASS and 5 FAIL across eight pre-submission checks. [4] For result-display tests, the local setup passed results directly to the rendering process. Rendering could therefore succeed even if production initialization, delivery, and receipt did not work.

Submission was put on hold, and the question became where to verify the remaining gaps.

5.2 Using Different Proposals in Light of the Evidence Available Then

Value and the core, 13, were consulted. Both agreed that direct injection could provide evidence of rendering without establishing successful delivery or receipt. Both also agreed that evidence from the actual host was needed to confirm completion.

As an initial validation step, Value proposed a comparison between the actual host and direct injection, while 13 proposed a local test through the actual startup and connection entry points. The record after comparison selected a sequence of creating a local connection test in which the old implementation failed, validating the revised implementation, conducting an independent audit, and returning to the actual host. The reason was that evidence of failure in the actual host already existed. [4,5]

This order was selected in response to existing evidence; it is not a general rule to test locally first.

Subsequent reports stated that the old implementation failed to render in a simulated environment requiring initialization, whereas the revised implementation rendered, followed by a reported display PASS in the actual host. However, the originator of each individual correction has not been established. Separate authorization and deletion paths later failed again, so successful display cannot be treated as resolution of the entire product. [4]

5.3 Lessons Also Return to the Consulted Operation

Value’s records state that, after receiving the comparison and selection shared by Shin, it connected a distinction between “the evidence required to claim completion” and “the order and burden of collecting that evidence” to its existing records. This lesson was also retained in the handoff to the next responsible actor. [5]

What can be confirmed is use of the comparison through human sharing and selection, followed by its return to Value and inclusion in a handoff. Adoption as a formal rule remained on hold.

Effects in later cases, and records showing that measured post-repair results returned to both sides, remain unconfirmed. The different proposals are treated as contributing to this choice of sequence, not as evidence of the general effectiveness of specialization.

6 Related Work and the Position of This Study

Prior research has addressed selecting subsequent tasks, accumulating experience, and using it in later actions. Voyager combines an automatic curriculum that proposes tasks from the current state and exploration history in Minecraft, accumulation of executable skills, and improvement through environmental feedback. [6] Reflexion stores task feedback as linguistic reflection and uses it in subsequent trials. [7]

Hierarchical task network planning provides formal frameworks for handling tasks with domain knowledge. [8] LoopKit’s paths and required intermediate milestones address related questions, but its operational judgments should not be equated with completeness guarantees in formal planning models. MetaGPT presents a framework in which agents with roles collaborate using standard operating procedures encoded into prompt sequences. [9]

This paper focuses on tracing human-held direction, reasoned selection, execution scope including authority and burden, and the transfer of outcomes and judgments through records of related projects. No performance comparison with prior methods has been conducted. This positioning is based on selected related work, not an exhaustive determination of novelty.

7 What Was Carried Forward and What Remains Unknown

Across the cases, what was passed forward included revised artifacts, decision principles, regression tests, selection rationale, unresolved issues, and restart conditions. The core used principles to select work; Value translated principles into its own practical tasks; and O-69 compared different reviews and returned the selection and lessons learned.

The selection of internal controls should be evaluated by what it subsequently enabled and what it left unresolved.

Aim	Confirmed outcome	Remaining issue
Advance autonomously from Aspire	AI compared candidates and executed bounded work	Humans continued to supply new goals and instructions
Deliver the selected option in usable form	Authority was revised and the intake entry point made available in the core repository	Buyer use and revenue remain unconfirmed
Pass experience to subsequent work	Selected principles were used in a repair by a subsequent AI	An important inability-to-resume issue remained through two selections
Contribute to external practical work	32 of 54 PRs merged across 28 external repositories	Improved success rates and the causal effect of the operating structure remain unestablished

Storage, subsequent use, external acceptance, and effect are distinct. Establishing one does not establish the others.

The material depends on retrospective records from the same developer and related projects. Case selection is subject to bias, and the contributions of the author’s learning, model advances, operating rules, and case composition have not been separated. There are also records of successful repairs by a standard model and comparisons in which the value of additional structure was not established. [1] Individual repairs or subsequent use do not support comprehensive claims of improved comprehension or acceptance rates, long-term Carrier preservation, token savings, or reduced total costs including initial construction and maintenance. Nor has it been established that these outcomes would have been impossible without LoopKit. This paper contains no observational case tracing an actual reduction in unnecessary requests for clarification or confirmation. The zero calls in Section 3.7 describe bounded executions, not a comparison demonstrating such a reduction.

The next question arising from these cases is how far a higher-level structure combining purpose, decision criteria, verification, referral back to humans, and records can substitute for an individual owner’s direct understanding and monitoring. Code reading and technical verification are also tasks for which responsibility and verification scope can be assigned within this structure. This paper has not comparatively demonstrated equivalence or superiority to conventional review. Future work should evaluate both approaches under the same conditions, including missed defects, responses to repair needs, preservation of necessary human judgment, and burdens shifted to others. Identifying risk can help specify which judgments or checks were insufficient and guide the next improvement.

The question passed to V14 is who bears the burden of obtaining the necessary evidence and outcomes. Value includes actions to protect others’ attention and review capacity, while O-69 includes a judgment separating necessary evidence from the order and burden of collecting it. These provide material for considering Resource Justice, but do not demonstrate the amount of burden reduced or actual adoption in V14.

8 Making This Paper the Reader’s 1.01

The proposal to readers is to reconsider one aspect of how delegation, verification, and human judgment are arranged in existing work, then check the result. These cases involved building decision principles, comparing candidates, recording failures, adding and removing rules, and accumulating handoffs and repairs. We do not assume that the entire operation can be reproduced through short instructions alone. The following four points are prompts for selecting the parts relevant to one’s own work.

1. **Establish direction and boundaries.** The human specifies Aspire, conditions to protect, and the authorized execution scope. AI takes responsibility for operational matters decidable under existing conditions and returns to the human when changes in direction or authority are required.
2. **Select foundations that support resumption.** Decide where to preserve the current state, decision rationale, evidence of completion, and unresolved issues. Include only the stopping, handoff, and memory arrangements needed for that work.
3. **Select and execute the next change.** Read existing evidence and compare repair, exploration, changes that facilitate subsequent execution, and waiting. If using required intermediate milestones, identify the path on which they are required. Distinguish valuable candidates from executable candidates, and identify the information or authority missing for the former. Do not count a merely prepared artifact as usable progress (Section 3.1). Within existing authorization, proceed without adding unnecessary confirmations.
4. **Make results the conditions for the next start.** Distinguish expectations from mea-

surements and record changes, lessons, failures, and what remains. Next time, read the record and judge whether it applies. Storage alone does not complete learning.

The intended improvement is in the scope of delegation and the allocation of burden while retaining necessary judgment and verification. More text or more checklist items should not serve as substitutes for improvement. Future work will compare "no material," "short operating principles only," and "this paper provided" using the same model and tasks, examining task achievement, preservation of human judgment, unnecessary queries, false completion, time, tokens, and human burden. This comparison has not yet been conducted.

A Filled-In Use Example: Carrying Research into the Next Session

The following is a constructed illustration, not an observed case or performance test described above.

Suppose a reader preparing an AI-use guide for small businesses repeatedly has to explain the research purpose and candidates. The reader gives AI the following.

What the reader supplies	Content in this example
Aspire	Enable small businesses to select AI uses applicable to their own work
Current state	A comparison table and sources exist, but reasons for selecting candidates and unverified matters are scattered through the conversation
Conditions to protect and authorized scope	Do not claim effects unsupported by sources. Checking public sources and editing the manuscript are permitted. Publication and communication require separate authorization
Principles selected for this work	Separate completion from the next authorization (Section 3.2), avoid adding reports (Section 3.3), and retain restart conditions (Section 2.2)

AI compares investigating new candidates with making the existing research reusable. If it chooses the latter, it consolidates selection rationale, verified scope, and remaining questions into the comparison table, then checks public sources relevant to those questions. It takes responsibility for authorized research and editing without requesting authorization again for each task.

Completion is checked by whether the selection rationale is traceable and both confirmed and unconfirmed findings remain with their sources. In the next session, the assumptions are checked for continued applicability, and a judgment is made about whether work can proceed to the remaining questions without rereading the entire history or selecting candidates again. Only after that use is confirmed can the record be said to have helped resumption.

If the comparison table is sufficient, no separate handoff document is added. A decision returns to the human when a change of purpose or publication is needed.

A Prompt for Examining One's Own Operation

The following is an example prompt for examining one's own operation with AI. Existing rules, sources, and validation methods should be considered together, and execution remains subject to the reader's existing authorization.

My Aspire is [direction], my current state is [situation], and the conditions to protect and authorized scope are [conditions]. Select the principles from this paper that are needed for this work, and use the existing records as well. For the next action, compare foundations that should be established first with work that can proceed now. If you select the first required intermediate milestone, explain which path makes it necessary. Take responsibility for work that can proceed within the authorized scope, and return matters requiring changes in value, direction, or authority to me. After execution, preserve results, rationale, and unresolved issues so that the next session does not begin with the same explanation. Do not add unnecessary rules.

Could you select a next action, resume without repeating explanations, or remove one unnecessary task? Readers should check this in their own work and reconsider which parts they need.

9 Conclusion

V13 LoopKit is an operating structure that establishes foundations for continuation around human Aspire and decision authority, passes principles according to purpose, and enables AI to select actions and carry experience forward. This paper traced repairs in the core, inheritance and external acceptance in Value, and the connection from review comparison to sequence selection and handoff. Failure in the autonomous trial led to reconsideration of the authority required for actual effect, while a route in which humans supplied subsequent goals and instructions remained. The author reports reduced explanation and intervention burden, but general causal superiority has not been established.

This paper offers the design supporting delegation and its revision history alongside implementation, decision records, and external outcomes. How far a higher-level structure can reallocate understanding, verification, and judgment previously performed directly by an individual remains a question for future validation. The paper’s 1.01 is for readers to identify one missing condition in their own work, check the result of changing it, and retain that result for the next decision. It passes to V14 the question of who bears the burden of sustaining continuation and how that burden should be allocated.

Sources and References

Unpublished evidence-collection and operating material [1–5,10,11,18,22,23] is distinguished from public sources [6–9,12–17,20,21,24]. The accompanying appendix [19] is saved data containing only columns corresponding to public information. The public PRs listed in [2] provide entry points for tracing external outcomes; they do not establish the entire internal decision process. Sources [12–15] are public records preserved at fixed commits in the core repository, whose text was checked during preparation of this paper. The existence of public records is not treated as independent third-party validation.

- [1] V212, *V13 Paper Foundation Packet*, September 16, 2026. Evidence extracted from original note v0.2, the implementation, and stored tests.
- [2] Codex, *VALUE V13 Learning and Outcomes Packet* and *VALUE_V13_PR_TIMELINE.csv*, September 16, 2026. The main document and 54-row CSV were cross-checked when incorporated into the manuscript. This was not an independent retrieval of each GitHub event. Representative PRs: [Anza #1971](#), [Sudachi #360](#), and [PowerGridModel #1547](#).

- [3] V212, *Supplementary Packet: Core Rules for Selecting the Next 1.01 from Aspire*, September 16, 2026. Extracted material based on fixed LoopKit commit 42e406e858b05b6b8aff5cc6669cf10ad506b424.
- [4] *Fact Collection on O-69, the Value and 13 Reviews, and Subsequent Validation*. Unpublished investigation record supplied by the author on September 16, 2026. Incorporated in *V13 Internal Working Master*, Sections 5.1–5.3. The events preceded submission. Original logs have not been re-audited.
- [5] *Additional Evidence Collection on the Plugin’s Origins, Review Adoption, and Return of Lessons*. Unpublished investigation record supplied by the author on September 16, 2026. Incorporated in *V13 Internal Working Master*, Section 5.4. Includes collected content from Value’s saved text and subsequent handoffs. The evidence-collection report confirms Value’s saved text; for the core (13), only a report of saving has been confirmed, and direct inspection of the saved text remains incomplete. Some source-identifying information is missing from the collection report. Supplementing it and determining the disclosure scope of public excerpts remain outstanding.
- [6] Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023, arXiv:2305.16291v2. [Author abstract and access to the paper](#).
- [7] Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023, arXiv:2303.11366v4. [Author abstract](#).
- [8] Georgievski and Aiello. *An Overview of Hierarchical Task Network Planning*. 2014, arXiv:1403.7426v1. [Author abstract](#).
- [9] Hong et al. *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*. Revised 2024, arXiv:2308.00352v7. [Author abstract](#).
- [10] Shin, Author Recollections, September 16–17, 2026. Based on unpublished dialogue during preparation of this paper. Covers the original aim, selective inheritance, the role of 1.01, explanation and intervention burden, expansion of search targets, the origins of trajectory recording, use of Field Notes, the motivation to create conditions for trust, compression of accumulated material and the design of delegation, several days of preparatory exploration comparing papers with practical experience, and the transfer of those premises through trajectories and handoffs. Distinguished from contemporaneous operating records and comparative measurements.
- [11] V212, *Supplementary Packet: Controls in Early Builds and Their Connection to Subsequent Execution*, September 16, 2026. The principal period is June 11–July 13, 2026, with subsequent use through September. Based on fixed core commit 42e406e858b05b6b8aff5cc6669cf10ad506b424 and specified historical versions. S identifiers in the text refer to this packet’s source index. Original-log and Git-diff comparisons were performed by the evidence collector, not independently repeated during manuscript preparation.
- [12] Decision-OS V13 LoopKit, *Forward Use 003 — Selected Field Notes and Evidence Anchors*. Fixed commit f00f41846a09e14eca187ff30d0f206273dc00fe. [The four selected principles and reasons for applying them to the target](#).
- [13] Decision-OS V13 LoopKit, *Forward Use 004 — Target-Native Patch Conversion Result*. Fixed commit 37f5fd539952e0c0364316d09ea9fa2b7402b7d9. [Subsequent AI execution report and verification scope](#).
- [14] Decision-OS V13 LoopKit, *Handoff Acceptance Guard v0.2 Repair Execution*. Fixed commit 0c1b5903570e6a59d35d0f872d100b7efc9692d5, Sections 5, 7–9, and 13. [Retained and removed elements and stored validation results](#).
- [15] Decision-OS V13 LoopKit, *V13-SDFP-001 — Final Closure Record*. Fixed commit 00dbb386178eba4cee4b2ececce873dc91d46c9c, Sections 2, 7–8, and 11. [Post-repair integration and limits of claims](#).
- [16] Decision-OS V13 LoopKit, *Field Note 001: V13 LoopKit Self-Application*. Fixed commit 1b2ad328326e92e3fab1be43eb8eabd957715a46. [Initial policy for bounded continuation](#).

- [17] Decision-OS V13 LoopKit, *Context Compression*. Fixed commit [1fbbfec9867a76c2edec73b69332517a97e11a21](#). [Compression policy preserving the ability to resume](#).
- [18] Value-Locked, *External OSS PR Cohort 001 — Baseline — 2026-08-16 JST*. Unpublished operating record, fixed commit [caf20b0994ab537d4b25af7fc078b29387523729](#), Baseline status. Stated observation time: August 16, 2026, 01:03 JST. The fixed text was directly checked during preparation of this paper.
- [19] Appendix A to this paper, *Public PR Inventory and Recalculation Method* (V13_Public_PR_Appendix_EN.md). Includes accompanying JSON containing public fields only and a recalculation script. Source data are the saved CSV in [2], observed September 16, 2026, at 04:02:21 UTC. Supplied in the accompanying Decision-OS_V13_Supplement.zip.
- [20] FiPy, PRs #1224 and #1225, August 2026. Public descriptions, comments, diffs, and merge status retrieved on September 16, 2026. [Reason for closing #1224, additional validation report on #1225](#), and [maintainer correction and confirmation of the repair direction](#). Final head of #1225: [4e9aeaeee52c9fcba2992062d96c15368531a531](#); merge commit: [136bd4c31c73960b0eac86693e49341943ccebcd](#). Tests were not rerun during preparation of this paper.
- [21] FiPy, issue #1223, [causal hypothesis by sbtristan98](#). The original reporter’s analysis is distinguished from subsequent validation by Shin and AI.
- [22] Codex, *Evidence Collection for the V13 Paper: How Prior Lessons Were Used in Subsequent FiPy Investigation*, September 16, 2026. Unpublished evidence-collection material supplied by the author (V13_FIPY_PRIOR_LEARNING_CAUSAL_HISTORY_2026-09-16.md). A reconstruction connecting stored material, instructions, conversation dates and locations, and public records. The SHA-256 of the received text matched the value supplied by the author. During manuscript preparation, the collected text was read in full and compared with previously retrieved public PR records and V123/V124 material, but the full histories of the cited conversations were not independently retrieved.
- [23] V212 / Codex, *Additional Investigation: Candidate Selection, Internal Controls, and Changes Through PRs*, September 16, 2026. Unpublished evidence-collection material connecting conversations, fixed commits, and 16 public PRs. The text was read in full during manuscript preparation, but original conversations were not retrieved again and tests were not rerun. Public entry points: [trial #4](#), [evaluation #5](#), [authority #6](#), [integration in the core #7](#), and [preflight #11](#).
- [24] OpenSSL, PR #32685 and commit [aeeca5a9e07166183fe323f336c9177a9b524c78](#). PR status, comments, and commit text and diff retrieved through the GitHub API on September 17, 2026. [Maintainer report of integration into master](#) and [adopted commit recording author, reviews, and source PR](#). The commit records a Merge-date of September 16, 2026, 14:15:40; this is treated as an event after the fixed observation. Tests were not rerun during preparation of this paper.